Day 37 – Tree Traversal (Inorder, Preorder, Postorder)

🌳 Tree Traversal Kya Hota Hai?

Tree traversal ka matlab hota hai:

👉 Tree ke har node ko ek specific order me visit karna

Binary Tree me mainly 3 types hote hain:

Inorder

Preorder

Postorder

🔢 Example Tree

Hum is tree ko use karke samjhenge:

```
    1
   / \
  2   3
 / \
4   5
```

1️⃣ Inorder Traversal (Left → Root → Right)

👉 Steps:

- ✓ Pehle left subtree
- ✓ Phir root
- ✓ Phir right subtree

👉 Flow:

4 → 2 → 5 → 1 → 3

👉 Trick:

💡 Left sabse pehle aata hai

2️⃣ Preorder Traversal (Root → Left → Right)

👉 Steps:

- ✓ Pehle root
- ✓ Phir left subtree
- ✓ Phir right subtree

👉 Flow:

1 → 2 → 4 → 5 → 3

👉 Trick:

💡 Root sabse pehle

3️⃣ Postorder Traversal (Left → Right → Root)

👉 Steps:

- ✓ Pehle left subtree
- ✓ Phir right subtree
- ✓ Last me root

👉 Flow:

4 → 5 → 2 → 3 → 1

👉 Trick:

💡 Root sabse last me

🔥 Recursive Code (Java)

```
Node Structure
 class Node {
    int data;
    Node left, right;

    Node(int data) {
        this.data = data;
        left = right = null;
    }
}
```

✅ Inorder

```
void inorder(Node root) {
    if (root == null) return;

    inorder(root.left);
    System.out.print(root.data + " ");
    inorder(root.right);
}
```

✅ Preorder

```
void preorder(Node root) {
    if (root == null) return;

    System.out.print(root.data + " ");
    preorder(root.left);
    preorder(root.right);
}
```

✅ Postorder

```
void postorder(Node root) {
    if (root == null) return;

    postorder(root.left);
    postorder(root.right);
    System.out.print(root.data + " ");
}
```

🧠 Easy Memory Trick

Traversal	**Order**
Inorder	L → R → R
Preorder	R → L → R
Postorder	L → R → R (Root last)

👉 Better yaad karne ka shortcut:

Inorder → Left Root Right

Preorder → Root Left Right

Postorder → Left Right Root

🌳 Complete Java Code (All Traversals Together)

```java
package Day_37;

class TreeTraversal {

    // Node class
    static class Node {
        int data;
        Node left, right;

        Node(int data) {
            this.data = data;
            left = right = null;
        }
    }

    // Inorder Traversal (Left, Root, Right)
    static void inorder(Node root) {
        if (root == null) return;

        inorder(root.left);
        System.out.print(root.data + " ");
        inorder(root.right);
    }

    // Preorder Traversal (Root, Left, Right)
    static void preorder(Node root) {
        if (root == null) return;

        System.out.print(root.data + " ");
        preorder(root.left);
        preorder(root.right);
    }

    // Postorder Traversal (Left, Right, Root)
    static void postorder(Node root) {
        if (root == null) return;

        postorder(root.left);
        postorder(root.right);
        System.out.print(root.data + " ");
    }

    public static void main(String[] args) {

        // Tree create kar rahe hain
        /*
                1
               / \
              2   3
             / \
            4   5
        */

        Node root = new Node(1);
        root.left = new Node(2);
        root.right = new Node(3);
        root.left.left = new Node(4);
        root.left.right = new Node(5);

        // Output
        System.out.print("Inorder: ");
        inorder(root);

        System.out.print("\nPreorder: ");
        preorder(root);

```

```
67.         System.out.print("\nPostorder: ");
68.         postorder(root);
69.     }
70. }
71.
```

🎯 Interview Tips

- ✓ Inorder of BST → sorted output deta hai
- ✓ Preorder → tree copy banane me useful
- ✓ Postorder → delete tree / memory free